

Indexing

Weihai Yu

October 2019

Indexing

- speeds up searching
- **search key**: attribute or set of attributes for an index
- **index file**: consists of **index entries**
 - | | |
|------------------|-------------------|
| search-key value | pointer to record |
|------------------|-------------------|
 - fixed-length records
- typically much smaller than table file
 - smaller records
 - possibly less entries

Search types

- equality search
- range search
- multi-attribute search
- partial-key search on a search key with multiple attributes

Ordered and hash indices

- ordered
 - ordered entries, binary search possible
 - tree structure
 - supports all kinds of search
- hashing
 - only supports equality search on search key
 - can be very fast

Indexing can be expensive

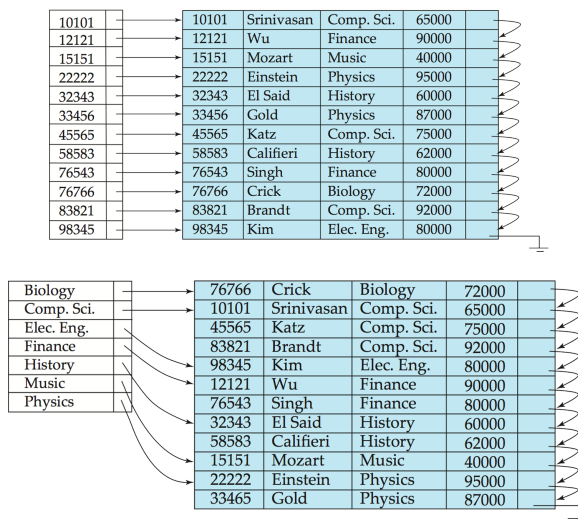
- extra disk and memory space
- must be updated when the table changes
- should be used when the benefit clearly outweighs the cost

Clustering, primary, secondary

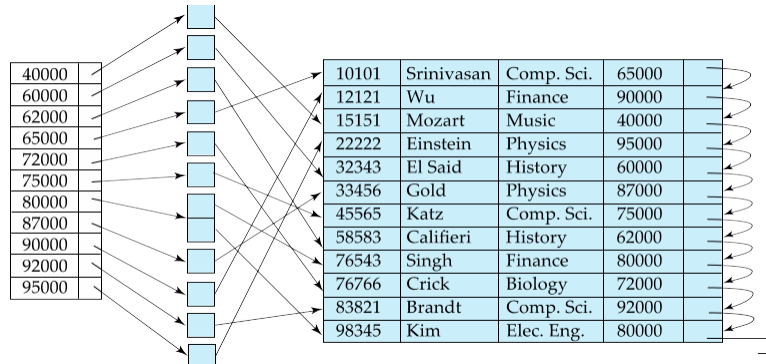
- **clustering**
 - index entries and table rows are ordered in the same way
 - there can be at most one clustering index on a table
 - also called a **primary index**
 - often on primary key
 - “create table” generally creates a primary index
 - **ordered sequential file**: ordered file with primary index
- otherwise, non-clustering, secondary

Dense index

every search-key value has an index entry

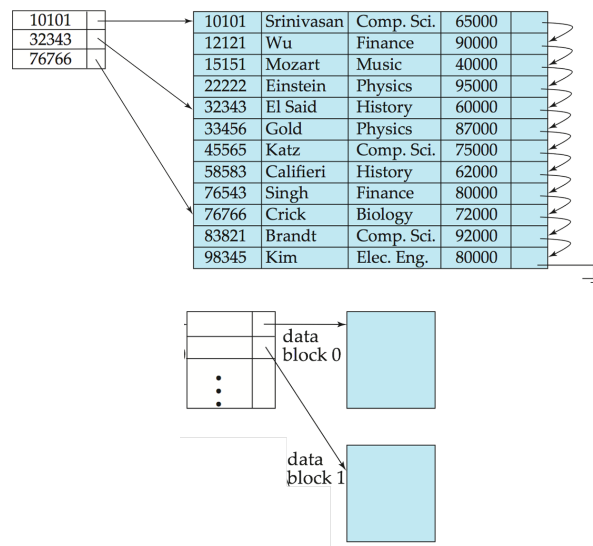


Dense index (non-clustering, non-key)



Sparse index

- only some search-key values have entries
- applicable for ordered files
 - clustering



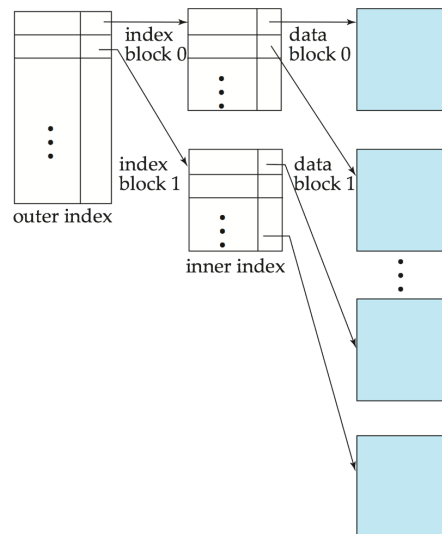
Example: cost of a range search

- data file has 10,000 blocks
- 20 records/block

- when indexed, 200 entries/block
- result has 100 rows (5 or **6** blocks)
- cost: **number of seeks** (10ms) + **number of block transfers** (0.1ms)
- for records
 - heap: **1***10ms + **10,000***0.1ms (1010ms)
 - file ordered on search key:
 $\log_2 10000 (\approx \mathbf{13}) * 10\text{ms} + (\mathbf{13} + \mathbf{6}) * 0.1\text{ms}$ (132ms)
 - non-clustering index: $\leq \mathbf{100}$ (1000ms)
 - clustering index: **1***10ms + **6***0.1ms (11ms)
- for index entries
 - non-clustering: $\log_2 1000 + (0 \text{ or } 1) \approx \mathbf{10 \text{ or } 11}$ (100ms) (all entries for rows in result must be read)
 - clustering (sparse index): $\log_2 50 (\approx \mathbf{6}) * 10\text{ms}$ (61ms)

Multilevel index

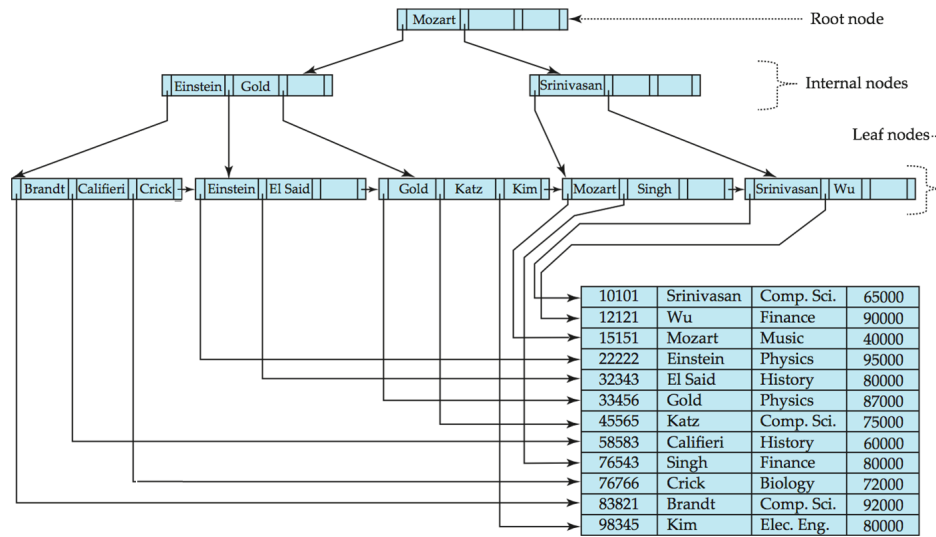
- index file may be too big
- index on index
 - clustering and sparse



Issue with ordered files for index entries

- insertion may create overflow blocks
- performance degrades as index file grows
- periodic file reorganization required

B⁺-tree index

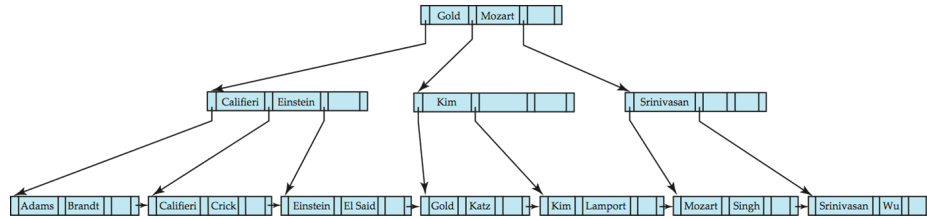


B⁺-tree index

- balanced tree
- one block for a node
- search-key values are ordered
- leaf nodes are linked
- node:

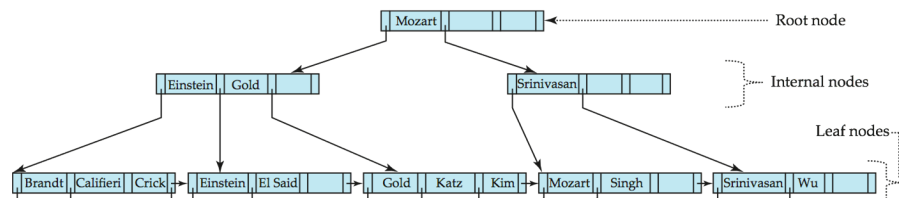
P_1	K_1	P_2	$\dots$	P_{m-1}	K_{m-1}	P_m
-------	-------	-------	---------	-----------	-----------	-------
- m: node *fanout*
- a B⁺-tree has a fixed n
 - a node has $\lceil \frac{n}{2} \rceil \leq m \leq n$ pointers
 - root: 2 (0 if also leaf) $\leq m \leq n$
- max tree height: $\log_{\frac{n}{2}} N$
 - if n= 100, for 1 million values, $\log_{50} 1000000 = 4$
 - compare with binary search, $\log_2 1000000 = 20$

Query with B⁺-tree index

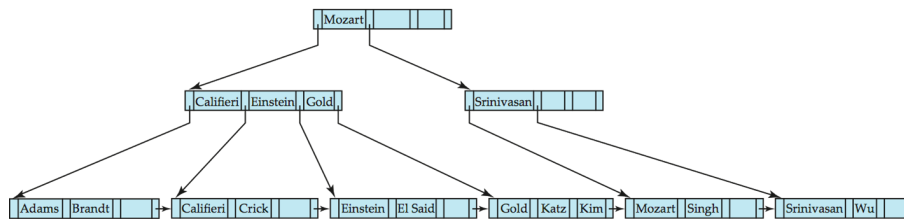


- navigate from root to leaf
- may need to get the sibling nodes
 - range search
 - when search key values are not unique

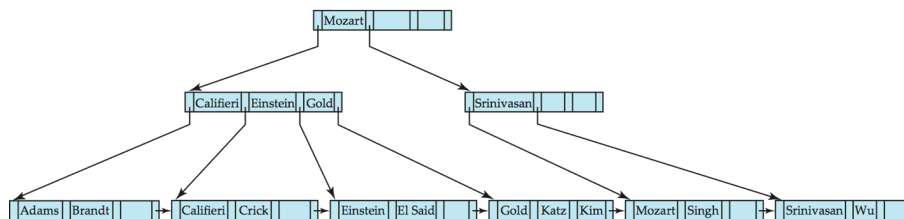
B⁺-tree insertion



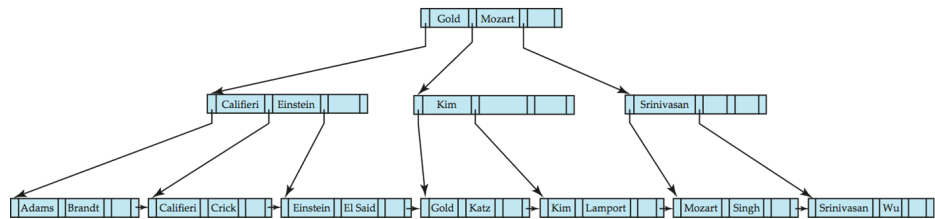
inserting “Adams”



B⁺-tree insertion

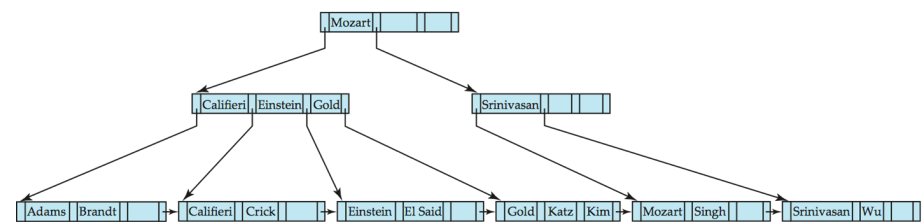


inserting “Lamport”

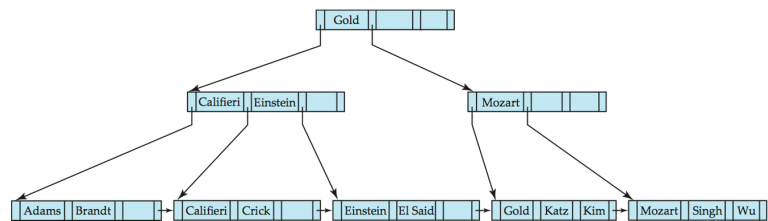


Notice the difference between splitting a leaf and a non-leaf!

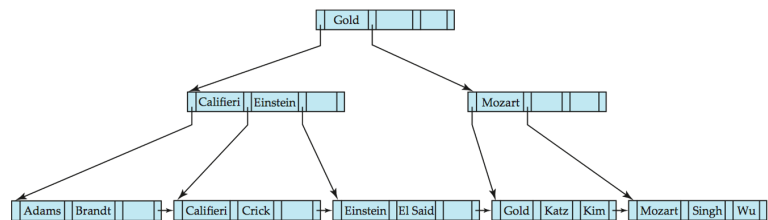
B⁺-tree deletion



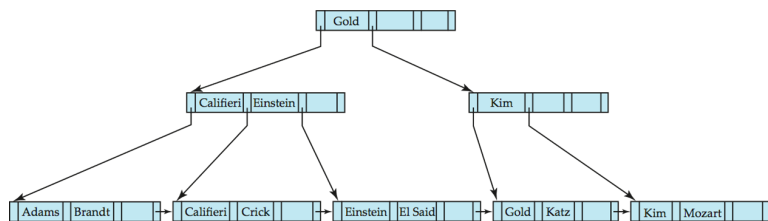
deleting “Srinivasan”



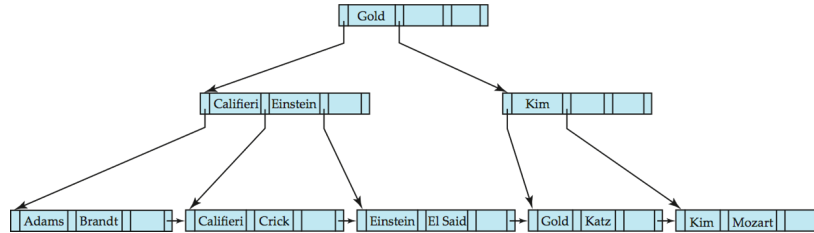
B⁺-tree deletion



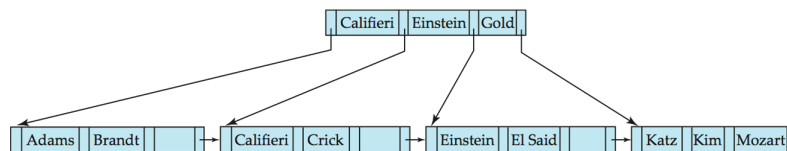
deleting “Singh” and “Wu”



B⁺-tree deletion

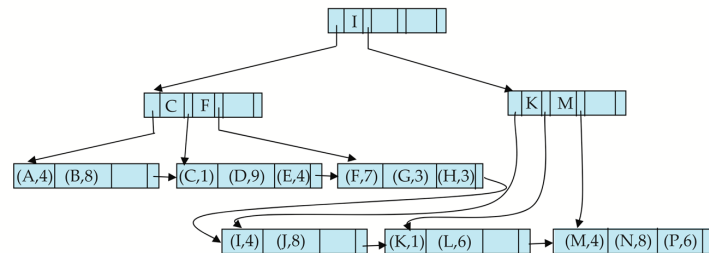


deleting “Gold”



Notice a bug in the figure ...

B⁺-tree variation: B⁺-tree file organization

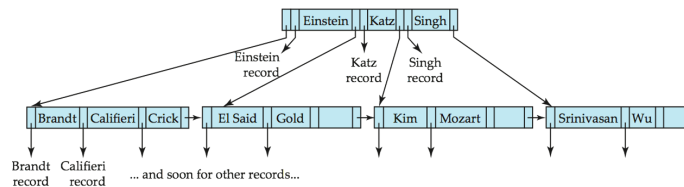


- Leaf nodes hold records rather than index entries
- space utilization of leaf nodes is more important
 - larger fanout, for example, between $\frac{2n}{3}$ and n
 - involve more sibling nodes for redistributing records

More B⁺-tree variations

- secondary index uses *primary keys* instead of pointers
 - for secondary index with lots of record movings
- string as search key
 - strings have long and variable length, but index entries should be short and fixed length

- *prefix compression*: for “Silberschatz”, use “Silb” if the closest values are “Silas” and “Silver”
- bulk loading: sort and insert
- B-tree index: a key value appears only once, non-leaf nodes point also to records
 - not used in practice. Why?



- Here are some of the reasons:
 - Each search key value in non-leave keys must have two pointers. So non-leave nodes have less number of values and the tree is taller.
 - Leave and non-leave nodes have different formats, making the algorithm more complicated.
 - Reading a range of records is more expensive.

Multiple-attribute search keys

- search key is a *sequence* of attributes (A, B)
- index entries are lexically ordered
- supports equality search
 - select ... where A=a and B=b
- supports range search
 - select ... where A=a and B<b
 - but not: select ... where A>a and B=b
- supports partial-key search
 - select ... where A<a
 - but not: select ... where B=b

Hash file organization

- a *hash-organized file* consists of buckets
- a *bucket* is one or more blocks containing records
- a *hash function* $b = h(v)$ calculates the address of bucket from the search key value
- all records in the same bucket have the same hash value on search key
- example
 - 10 buckets
 - hash function of char strings:
sum of binary representation of string modulo 10
 - $h(\text{Mary}) = 1$, $h(\text{John}) = 3$, $h(\text{Tom}) = 3$

Example of hash file organization

bucket 0

bucket 1

15151	Mozart	Music	40000

bucket 2

32343	El Said	History	80000
58583	Califieri	History	60000

bucket 3

22222	Einstein	Physics	95000
33456	Gold	Physics	87000
98345	Kim	Elec. Eng.	80000

bucket 4

12121	Wu	Finance	90000
76543	Singh	Finance	80000

bucket 5

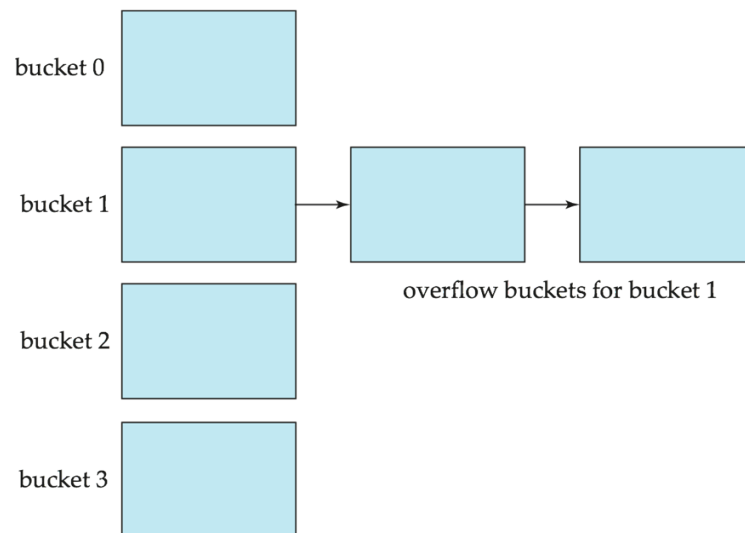
76766	Crick	Biology	72000

bucket 6

10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

bucket 7

Bucket overflow

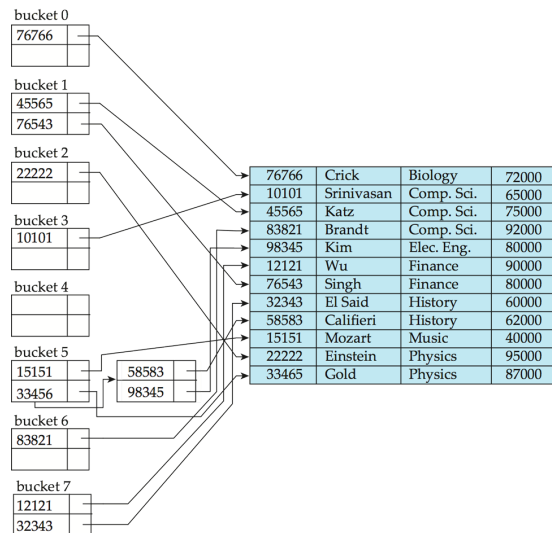


Searching

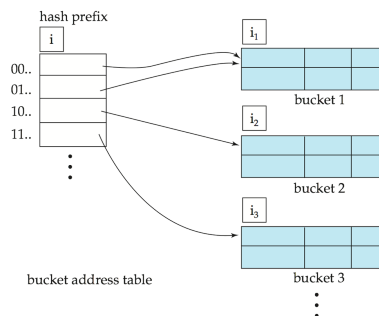
- calculate the hash value of the search key
- find the first block of the bucket
- walk through all blocks of the bucket
 - including the overflow buckets
- if a bucket consists of b blocks and n overflow buckets
- the cost is $n + 1$ seeks and $(n + 1)b$ block transfers

Hash index

- hash index is always secondary



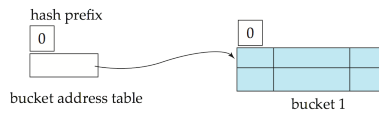
Extensible hashing



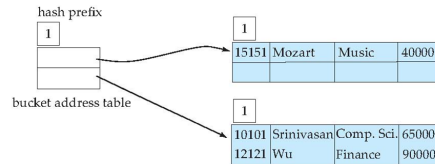
- hash function generates large range of values, typically 32 bits
- at any time, only prefix i bit values are used to index into a (level- i) *bucket address table*
- a bucket j has a level i_j ($i_j \leq i$)

Extensible hashing

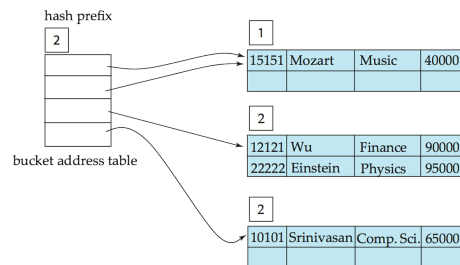
dept_name	h(dept_name)
Biology	0010...
Comp.Sci.	1111...
Elec.Eng.	0100...
Finance	1010...
History	1100...
Music	0011...
Physics	1001...



insert (Srinivasan, Comp.Sci.), (Wu, Finance), (Mozart, Music)

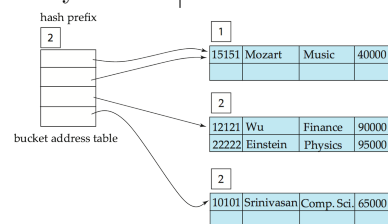


insert (Einstein, Physics)

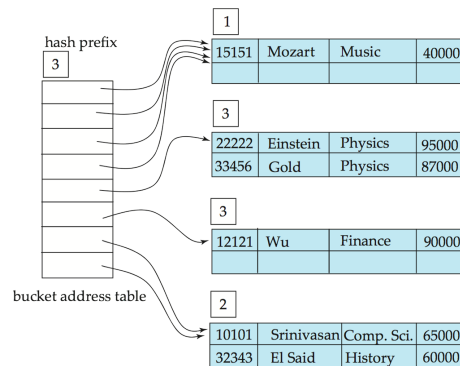


Extensible hashing

dept_name	h(dept_name)
Biology	0010...
Comp.Sci.	1111...
Elec.Eng.	0100...
Finance	1010...
History	1100...
Music	0011...
Physics	1001...

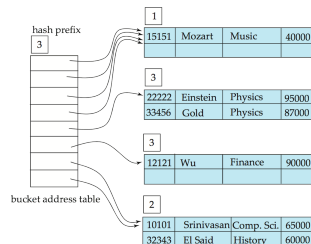


insert (El Said, History.), (Gold, Physics)

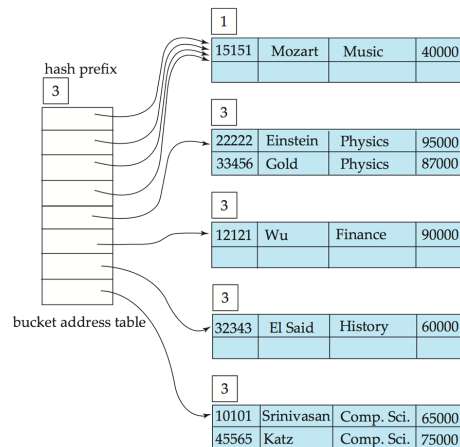


Extensible hashing

dept_name	h(dept_name)
Biology	0010...
Comp.Sci.	1111...
Elec.Eng.	0100...
Finance	1010...
History	1100...
Music	0011...
Physics	1001...

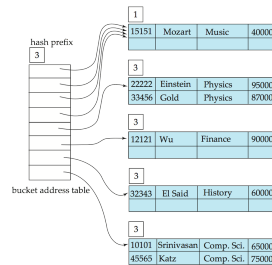


insert (Katz, Comp.Sci.)

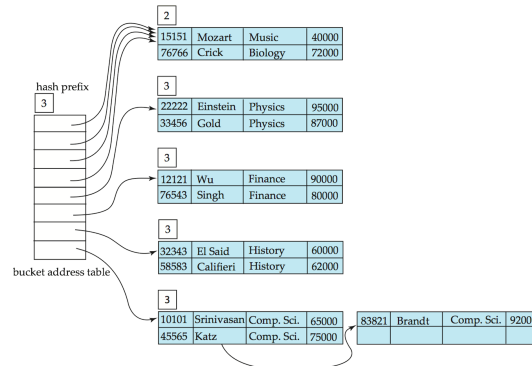


Extensible hashing

dept_name	h(dept_name)
Biology	0010...
Comp.Sci.	1111...
Elec.Eng.	0100...
Finance	1010...
History	1100...
Music	0011...
Physics	1001...



insert (Califieri, History), (Singh, Finance), (Crick, Biology), (Brandt, Comp.Sci.)



Extensible hashing evaluation

- Hash performance does not degrade with growth of file
- Reasonable space overhead
- extra level of indirection
- bucket address table may be large (larger than memory)
 - $2^{32} \approx 4$ giga, $2^{64} \approx 184$ peta

Choosing an index

- Hashing generally performs better for equality search
 - on average $\sim 1.2 \times b$ disk access (b bucket size)
 - particularly when table sizes are stable, have plenty of disk space, searches are selective (for hash index)
- B⁺-tree index supports *all* search types: equality, range, multiple attributes, partial-key, ...

File organization (again)

- unordered
- ordered
 - fill factor
 - overflow blocks
- B⁺-tree organized
- hash-organized

An index file can be

- linearly ordered
- B⁺-tree
- hash-organized

the last two are also called files with integrated indexing